

Syed Ammar Ali Zaidi – 26964 – OS HW1

Branch: hw1

Repo: <https://github.com/<yourname>/xv6-public>

Q2.

Linux (ELF + execve path)

- Process is usually fork() then execve(path, argv, envp).
- Kernel validates ELF, maps PT_LOAD segments, sets up user VA space + stack.
- Places argc/argv/envp and auxv on the initial stack; maps vDSO/vvar.
- If dynamically linked, PT_INTERP points to ld-linux; loader resolves DSOs, relocations, TLS.
- __libc_start_main runs constructors then calls main(); atexit handlers on exit.

Windows (PE + CreateProcess path)

- CreateProcessW creates EPROCESS + primary thread; user-mode stub in ntdll (RtlUserThreadStart).
- Loader maps PE sections, applies relocations, fixes Import Address Table; loads DLLs via ntdll.
- Security: DEP/ASLR/CFG; sets up TLS and runs TLS callbacks before entry.
- CRT entry (mainCRTStartup/WinMainCRTStartup) builds argv from Unicode command line, runs C++ static inits, calls main(); ExitProcess handles teardown.

Key contrasts

- Binary/loader: ELF+ld.so vs PE+ntdll/Kernel32.
- Startup data: Linux passes argv/envp/auxv on the stack; Windows provides a Unicode command line and environment block, CRT builds argv.

- Syscalls: Linux uses syscall gate directly; Windows apps call Win32 API which transitions via ntdll to NT syscalls.

Q3.

main() → loop:

getcmd(buf) reads a line and handles history/!!/!n.

parsecmd(buf) builds a command tree.

fork(); in child: runcmd(tree).

runcmd handles nodes:

exec → exec()

redir → open/dup then recurse

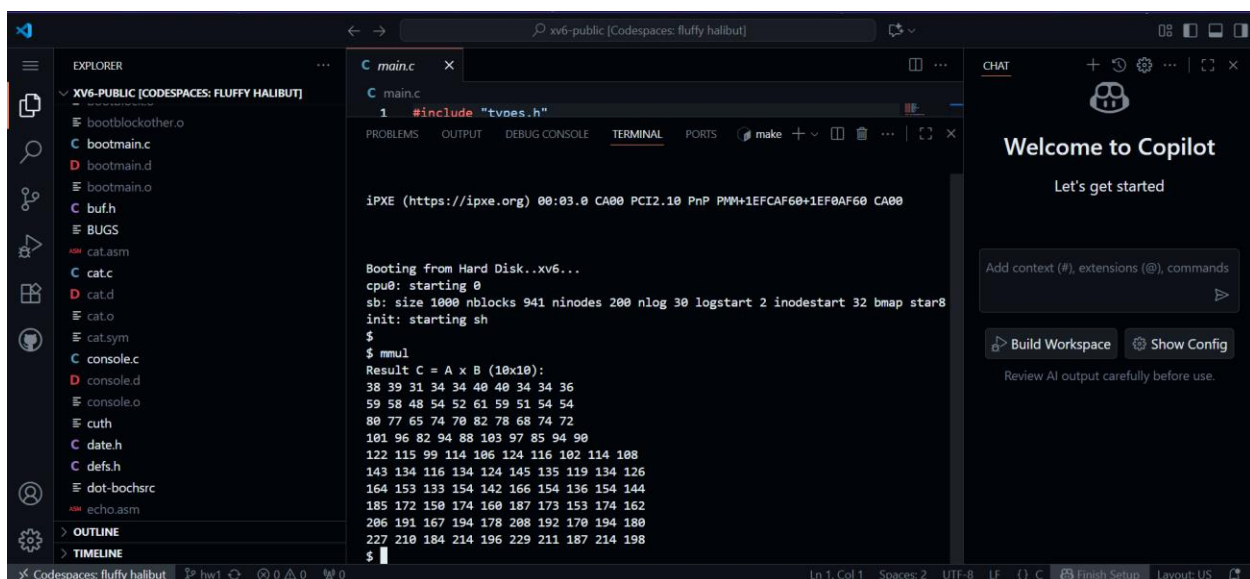
pipe → pipe(), fork left/right, dup ends, wait

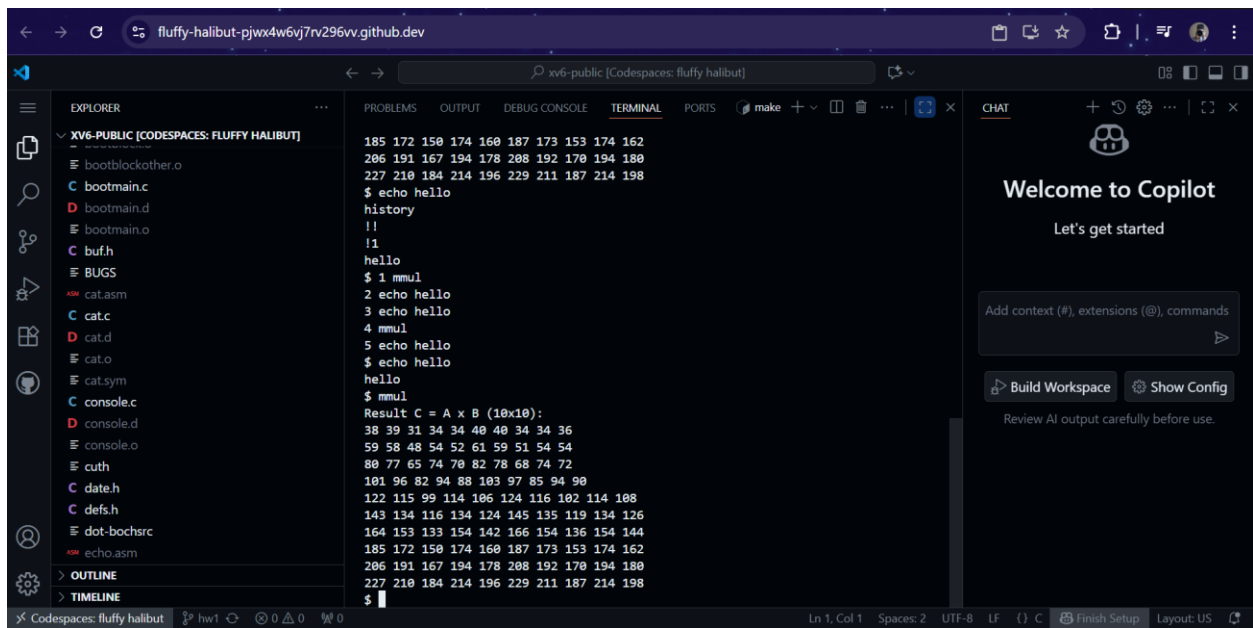
list → run left then right (A; B)

back → run without waiting (A &)

Parent waits for foreground jobs with wait().

Screenshots:





Build/Run commands

make clean && make qemu-nox

inside xv6:

mmul

echo hi

history

!!

!1